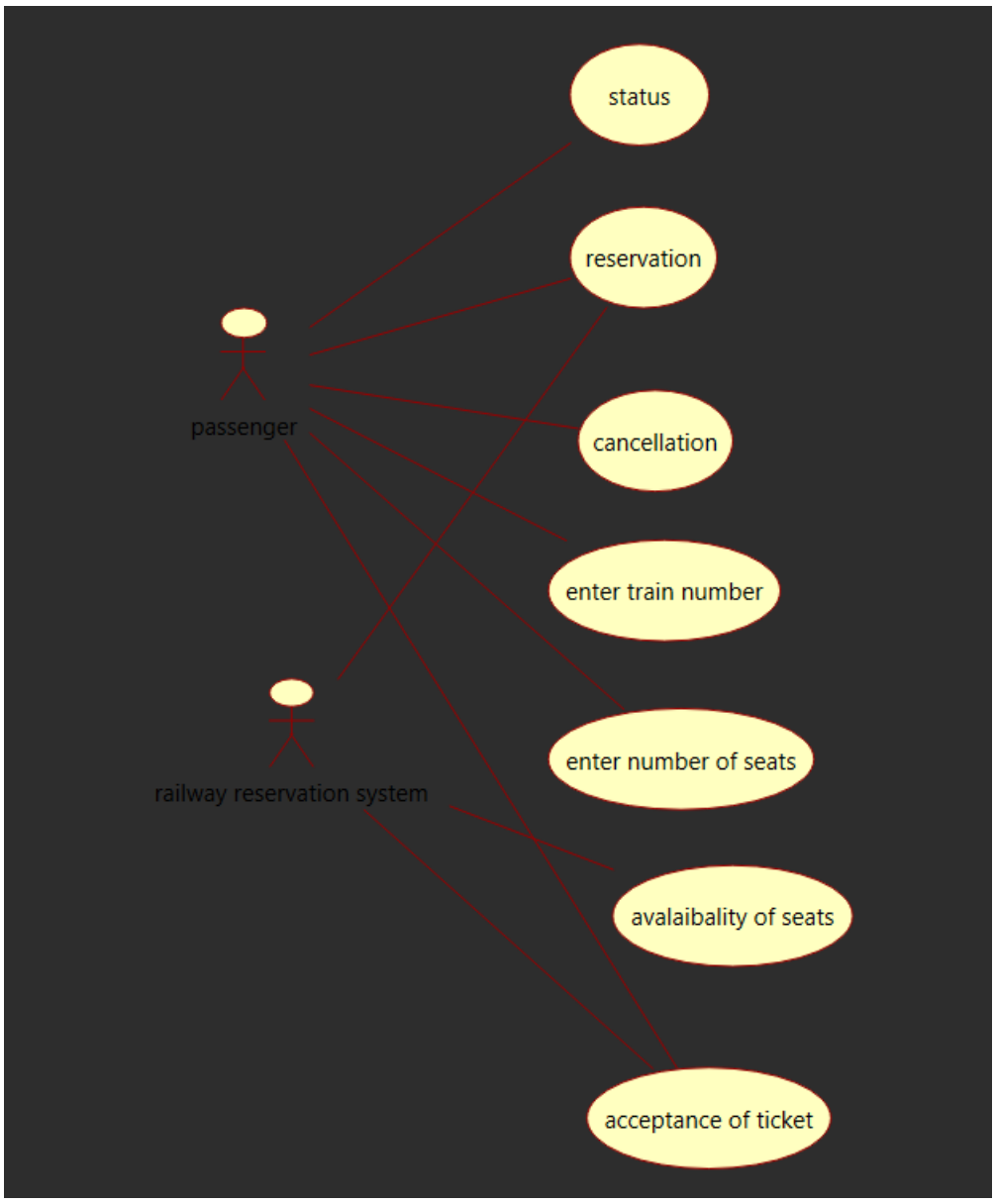
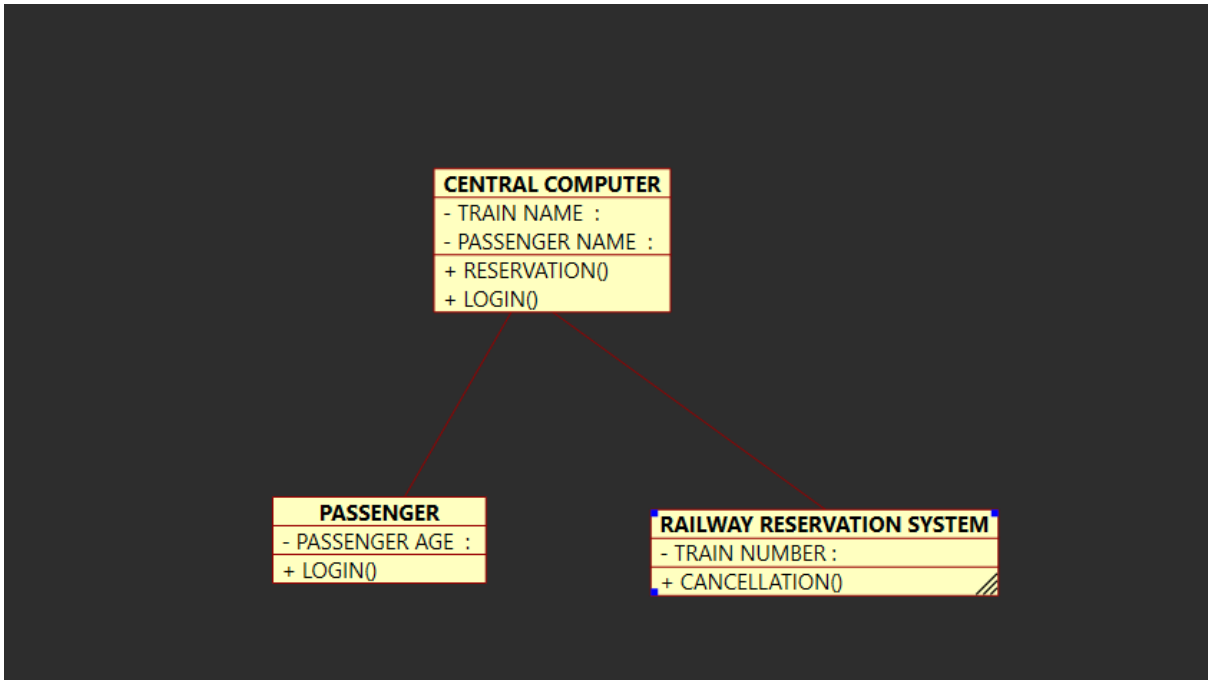
CSA1110- OBJECT ORIENTED ANALYSIS & DESIGN

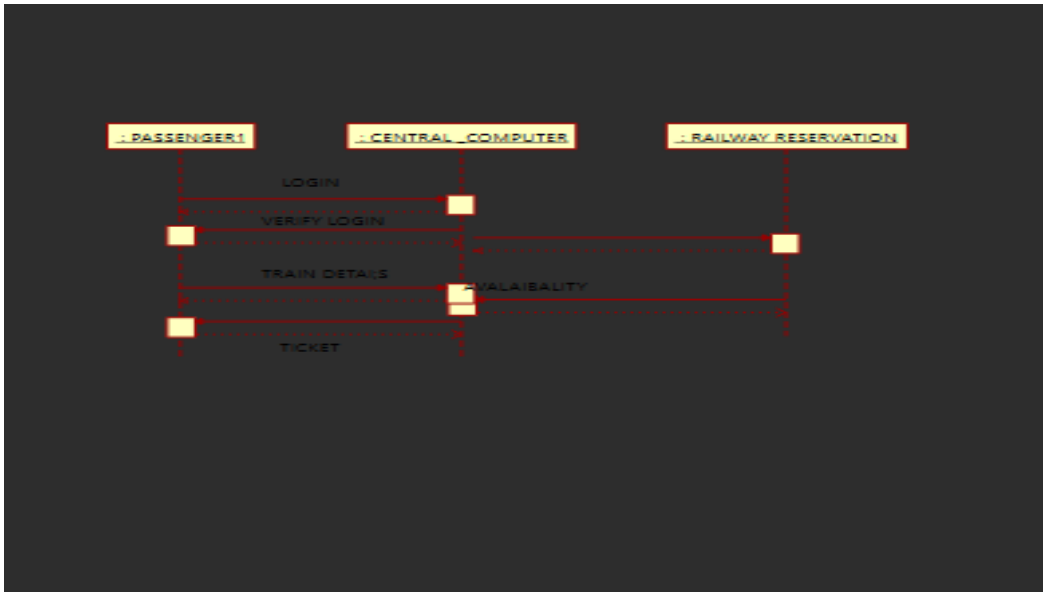
EXP -6 E – TICKETING

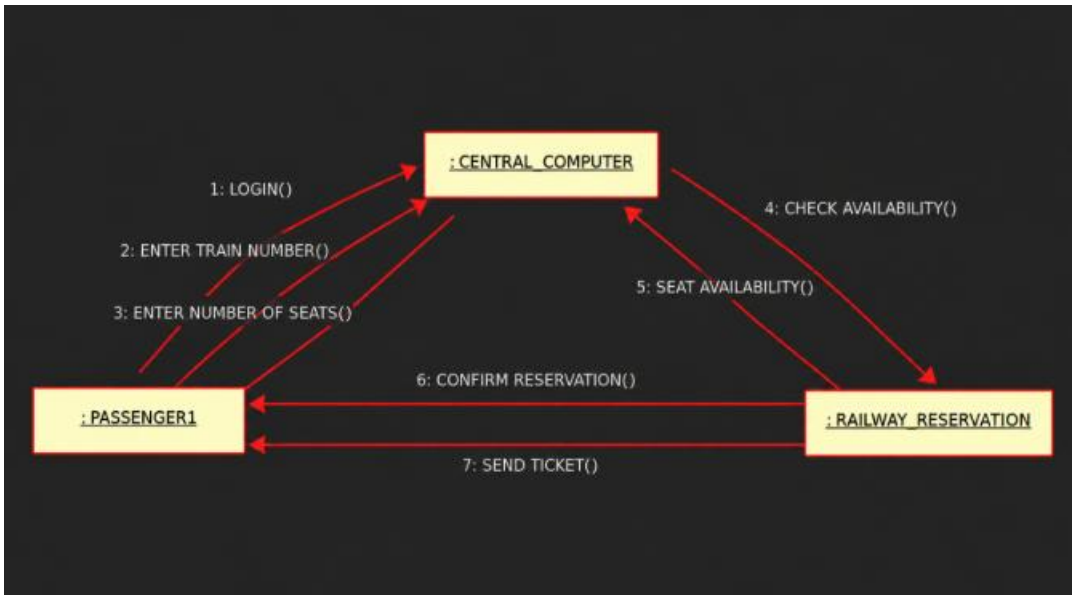
UML DIAGRAMS

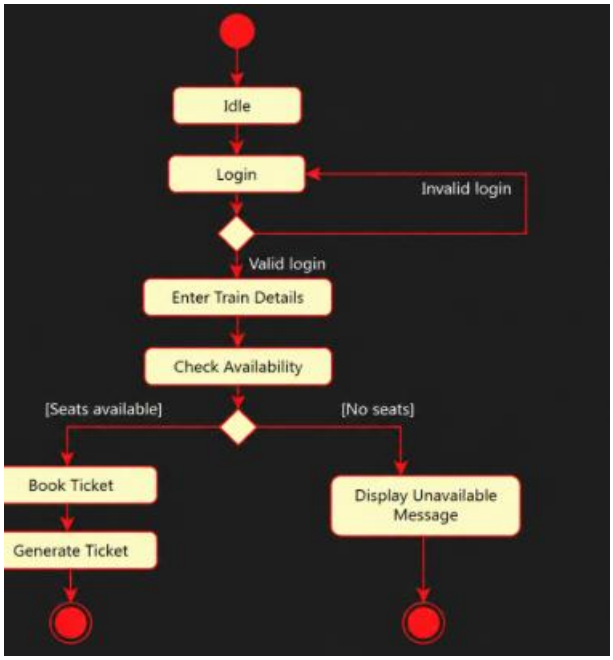


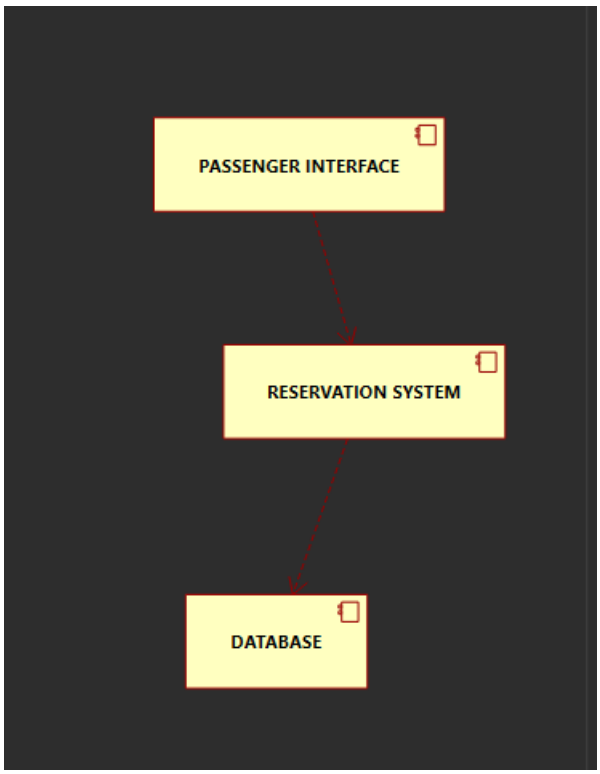

CENTRAL COMPUTER
- TRAIN NAME :
- PASSENGER NAME :
+ RESERVATION()
+ LOGIN()
PASSENGER
- PASSENGER AGE :
+ LOGIN()
RAILWAY RESERVATION SYSTEM
- TRAIN NUMBER :
+ CANCELLATION()


LOGIN TO SYSTEM
VERIFY LOGIN
ENTER PASSENGER DETAIKS
SELECT TRAIN
ENTER TRAIN NUMBER
ENTER NUMBER OF SEATS
CHECK AVALAIBALITY
BOOK TICKET
DISPLAY ERROR
SEND TICKET


: PASSENGER1
: CENTRAL _COMPUTER
: RAILWAY RESERVATION
LOGIN
VERIFY LOGIN
TRAIN DETAILS
TICKET


: CENTRAL_COMPUTER
1: LOGIN()
2: ENTER TRAIN NUMBER()
3: ENTER NUMBER OF SEATS()
4: CHECK AVAILABILITY()
5: SEAT AVAILABILITY()
6: CONFIRM RESERVATION()
: PASSENGER1
: RAILWAY_RESERVATION
7: SEND TICKET()


Idle
Login
Invalid login
Valid login
Enter Train Details
Check Availability
[Seats available]
[No seats]
Book Ticket
Generate Ticket
Display Unavailable Message


PASSENGER INTERFACE
RESERVATION SYSTEM
DATABASE


PASSENGER DEVICE
CENTRAL COMPUTER
RAILWAY RESERVATION DATABASE

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "1234",
    "database": "eticketing",
    "port": 3306,
}
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector
from mysql.connector import Error

# EDIT THIS PASSWORD before running.
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "YOUR_MYSQL_PASSWORD",
    "database": "eticketing",
    "port": 3306,
}

BG, PANEL, PANEL2 = "#10151f", "#182131", "#202c40"
TEXT, MUTED, ACCENT = "#eef3fb", "#a9b6ca", "#5b8cff"
GREEN, RED, YELLOW = "#2dc99a", "#ff6378", "#f6c85f"


def connect_db():
    return mysql.connector.connect(**DB_CONFIG)


def setup_database():
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""CREATE TABLE IF NOT EXISTS passengers (
        passenger_id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(50) UNIQUE NOT NULL,
        password VARCHAR(100) NOT NULL,
        passenger_name VARCHAR(100) NOT NULL,
        age INT)""")
    cur.execute("""CREATE TABLE IF NOT EXISTS trains (
        train_no INT PRIMARY KEY,
        train_name VARCHAR(100) NOT NULL,
        total_seats INT NOT NULL)""")
    cur.execute("""CREATE TABLE IF NOT EXISTS bookings (
        booking_id INT AUTO_INCREMENT PRIMARY KEY,
        passenger_id INT NOT NULL,
        train_no INT NOT NULL,
        seats INT NOT NULL,
        booking_status VARCHAR(20) DEFAULT 'BOOKED',
        FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id),
```

```
        FOREIGN KEY (train_no) REFERENCES trains(train_no))""")
    cur.execute("SELECT COUNT(*) FROM trains")
    if cur.fetchone()[0] == 0:
        cur.executemany("INSERT INTO trains VALUES (%s,%s,%s)", [
            (101, "Chennai Express", 100),
            (102, "Vellore Passenger", 80),
            (103, "Bangalore Express", 120)])
    conn.commit()
    cur.close()
    conn.close()


def rows(sql, params=()):
    conn = connect_db()
    cur = conn.cursor(dictionary=True)
    cur.execute(sql, params)
    result = cur.fetchall()
    cur.close()
    conn.close()
    return result


def run_sql(sql, params=()):
    conn = connect_db()
    cur = conn.cursor()
    cur.execute(sql, params)
    conn.commit()
    last_id = cur.lastrowid
    cur.close()
    conn.close()
    return last_id


def seats_left(train_no):
    train = rows("SELECT total_seats FROM trains WHERE train_no=%s", (train_no,))
    if not train:
        return None
    booked = rows("""SELECT COALESCE(SUM(seats),0) AS n FROM bookings
        WHERE train_no=%s AND booking_status='BOOKED'""", (train_no,))[0]["n"]
    return train[0]["total_seats"] - int(booked)


class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("E-Ticketing | Reservation Console")
        self.geometry("1180x740")
        self.minsize(980, 620)
        self.configure(bg=BG)
        self.selected_passenger = None
```

```
        self.selected_train = None
        self.selected_booking = None
        self.style_ui()
        self.make_shell()
        self.dashboard()

    def style_ui(self):
        s = ttk.Style(self)
        s.theme_use("clam")
        s.configure(".", font=("Segoe UI", 10))
        s.configure("Treeview", background=PANEL, foreground=TEXT,
                    fieldbackground=PANEL, rowheight=29, borderwidth=0)
        s.configure("Treeview.Heading", background=PANEL2, foreground=TEXT,
                    font=("Segoe UI", 10, "bold"), relief="flat")
        s.map("Treeview", background=[("selected", ACCENT)],
              foreground=[("selected", "#ffffff")])
        s.configure("TCombobox", fieldbackground=PANEL2, background=PANEL2,
                    foreground=TEXT, arrowcolor=TEXT)

    def shell(self):
        pass

    def make_shell(self):
        side = tk.Frame(self, bg="#0b1018", width=215)
        side.pack(side="left", fill="y")
        side.pack_propagate(False)
        tk.Label(side, text="◈  E-TICKETING", bg="#0b1018", fg=TEXT,
                 font=("Segoe UI", 16, "bold")).pack(anchor="w", padx=18, pady=(24, 3))
        tk.Label(side, text="RESERVATION CONSOLE", bg="#0b1018", fg=MUTED,
                 font=("Segoe UI", 8, "bold")).pack(anchor="w", padx=20, pady=(0, 24))
        for label, cmd in [
            ("▦  Dashboard", self.dashboard),
            ("♙  Passengers", self.passengers_page),
            ("▤  Trains", self.trains_page),
            ("▣  Book Ticket", self.booking_page),
            ("☷  All Bookings", self.bookings_page),
        ]:
            tk.Button(side, text=label, command=cmd, anchor="w", bg="#0b1018",
                      fg=TEXT, activebackground=PANEL2, activeforeground=TEXT,
                      relief="flat", padx=18, pady=13, font=("Segoe UI", 10, "bold"),
                      cursor="hand2").pack(fill="x", padx=7, pady=2)
        tk.Frame(side, bg="#273246", height=1).pack(fill="x", padx=18, pady=20)
        tk.Label(side, text="● MySQL live data", bg="#0b1018",
fg=GREEN).pack(anchor="w", padx=20)
        self.main = tk.Frame(self, bg=BG)
        self.main.pack(side="left", fill="both", expand=True)
        self.status = tk.StringVar(value="Ready")
        tk.Label(self, textvariable=self.status, bg="#0b1018", fg=MUTED,
                 anchor="w", padx=12, pady=5).pack(side="bottom", fill="x")
```

```
    def clear(self):
        for child in self.main.winfo_children():
            child.destroy()

    def heading(self, title, subtitle=""):
        f = tk.Frame(self.main, bg=BG)
        f.pack(fill="x", padx=25, pady=(22, 13))
        tk.Label(f, text=title, bg=BG, fg=TEXT, font=("Segoe UI", 23,
"bold")).pack(anchor="w")
        if subtitle:
            tk.Label(f, text=subtitle, bg=BG, fg=MUTED).pack(anchor="w", pady=(4, 0))

    def label(self, parent, text):
        return tk.Label(parent, text=text, bg=PANEL, fg=MUTED, font=("Segoe UI", 9,
"bold"))

    def entry(self, parent, width=20):
        return tk.Entry(parent, width=width, bg=PANEL2, fg=TEXT, insertbackground=TEXT,
                        relief="flat", highlightbackground="#34435b", highlightthickness=1,
                        font=("Segoe UI", 10))

    def button(self, parent, text, command, color=ACCENT):
        return tk.Button(parent, text=text, command=command, bg=color, fg="white",
                         activebackground=color, activeforeground="white", relief="flat",
                         padx=13, pady=8, font=("Segoe UI", 9, "bold"), cursor="hand2")

    def tree(self, columns, headings):
        frame = tk.Frame(self.main, bg=BG)
        frame.pack(fill="both", expand=True, padx=25, pady=(5, 20))
        tree = ttk.Treeview(frame, columns=columns, show="headings")
        for col, title, width in headings:
            tree.heading(col, text=title)
            tree.column(col, width=width, anchor="center")
        sb = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
        tree.configure(yscrollcommand=sb.set)
        tree.pack(side="left", fill="both", expand=True)
        sb.pack(side="right", fill="y")
        return tree

    def fail(self, error):
        messagebox.showerror("MySQL error", f"{error}\n\nCheck DB_CONFIG and ensure
MySQL is running.")
        self.status.set("Database error")

    def dashboard(self):
        self.clear()
        try:
            pc = rows("SELECT COUNT(*) n FROM passengers")[0]["n"]
            tc = rows("SELECT COUNT(*) n FROM trains")[0]["n"]
```

```
        bc = rows("SELECT COUNT(*) n FROM bookings WHERE
booking_status='BOOKED'")[0]["n"]
        available = sum(seats_left(t["train_no"]) or 0 for t in rows("SELECT train_no FROM
trains"))
        self.heading("Dashboard", "Live overview of your MySQL-backed reservation
system.")
        cards = tk.Frame(self.main, bg=BG); cards.pack(fill="x", padx=25)
        for title, val, color in [("Passengers", pc, ACCENT), ("Trains", tc, GREEN),
                                  ("Active bookings", bc, YELLOW), ("Seats available", available,
RED)]:
            c = tk.Frame(cards, bg=PANEL, padx=16, pady=15,
highlightbackground="#273246", highlightthickness=1)
            c.pack(side="left", fill="x", expand=True, padx=(0, 10))
            tk.Label(c, text=title.upper(), bg=PANEL, fg=MUTED, font=("Segoe UI", 9,
"bold")).pack(anchor="w")
            tk.Label(c, text=str(val), bg=PANEL, fg=color, font=("Segoe UI", 25,
"bold")).pack(anchor="w", pady=(5, 0))
        tk.Label(self.main, text="RECENT BOOKINGS", bg=BG, fg=TEXT,
                 font=("Segoe UI", 12, "bold")).pack(anchor="w", padx=25, pady=(24, 7))
        tree = self.tree(("id", "passenger", "train", "seats", "status"),
            [("id", "Booking ID", 90), ("passenger", "Passenger", 210),
             ("train", "Train", 250), ("seats", "Seats", 80), ("status", "Status", 120)])
        for r in rows("""SELECT
b.booking_id,p.passenger_name,t.train_name,b.seats,b.booking_status
            FROM bookings b JOIN passengers p ON p.passenger_id=b.passenger_id
            JOIN trains t ON t.train_no=b.train_no ORDER BY b.booking_id DESC LIMIT
10"""):
            tree.insert("", "end", values=(r["booking_id"], r["passenger_name"],
r["train_name"], r["seats"], r["booking_status"]))
        self.status.set("Connected to MySQL • Dashboard loaded")
    except Error as e: self.fail(e)

  # ---------- Passenger CRUD ----------
  def passengers_page(self):
    self.clear(); self.heading("Passenger Management", "Add, view, edit and delete
passenger records.")
    form = tk.Frame(self.main, bg=PANEL, padx=14, pady=13); form.pack(fill="x",
padx=25, pady=(0, 10))
    self.p_user, self.p_pass, self.p_name, self.p_age = [self.entry(form, w) for w in (16, 16,
21, 7)]
    for i, (title, widget) in enumerate([("Username", self.p_user), ("Password", self.p_pass),
("Full name", self.p_name), ("Age", self.p_age)]):
        self.label(form, title).grid(row=0, column=i, sticky="w", padx=5, pady=(0, 4));
widget.grid(row=1, column=i, padx=5, sticky="ew")
    a = tk.Frame(form, bg=PANEL); a.grid(row=2, column=0, columnspan=4, sticky="w",
pady=(11, 0))
    self.button(a, "＋ Add", self.add_passenger, GREEN).pack(side="left", padx=(0, 6))
    self.button(a, "Update Selected", self.update_passenger).pack(side="left", padx=6)
    self.button(a, "Delete Selected", self.delete_passenger, RED).pack(side="left", padx=6)
    self.button(a, "Clear", self.clear_p, PANEL2).pack(side="left", padx=6)
```

```
        self.ptree = self.tree(("id", "user", "name", "age"), [("id", "ID", 70), ("user",
"Username", 160), ("name", "Passenger Name", 280), ("age", "Age", 70)])
        self.ptree.bind("<<TreeviewSelect>>", self.select_passenger); self.refresh_passengers()

    def refresh_passengers(self):
        if not hasattr(self, "ptree") or not self.ptree.winfo_exists(): return
        self.ptree.delete(*self.ptree.get_children())
        for r in rows("SELECT passenger_id,username,passenger_name,age FROM passengers
ORDER BY passenger_id DESC"):
            self.ptree.insert("", "end", values=(r["passenger_id"], r["username"],
r["passenger_name"], r["age"]))

    def select_passenger(self, _=None):
        s = self.ptree.selection()
        if not s: return
        v = self.ptree.item(s[0], "values"); self.selected_passenger = int(v[0])
        self.clear_p(False)
        self.p_user.insert(0, v[1]); self.p_name.insert(0, v[2]); self.p_age.insert(0, "" if v[3] ==
"None" else v[3])

    def clear_p(self, reset=True):
        for w in (self.p_user, self.p_pass, self.p_name, self.p_age): w.delete(0, "end")
        if reset: self.selected_passenger = None

    def add_passenger(self):
        user, pw, name, age = self.p_user.get().strip(), self.p_pass.get(), self.p_name.get().strip(),
self.p_age.get().strip()
        if not user or not pw or not name: messagebox.showwarning("Required", "Username,
password and name are required."); return
        try:
            age = int(age) if age else None
            if age is not None and not 1 <= age <= 120: raise ValueError
            run_sql("INSERT INTO passengers(username,password,passenger_name,age)
VALUES(%s,%s,%s,%s)", (user,pw,name,age))
            self.clear_p(); self.refresh_passengers(); messagebox.showinfo("Added", "Passenger
added.")
        except ValueError: messagebox.showwarning("Age", "Age must be between 1 and
120.")
        except Error as e: self.fail(e)

    def update_passenger(self):
        if not self.selected_passenger: messagebox.showwarning("Select", "Select a passenger
row first."); return
        user, pw, name, age = self.p_user.get().strip(), self.p_pass.get(), self.p_name.get().strip(),
self.p_age.get().strip()
        if not user or not name: messagebox.showwarning("Required", "Username and name are
required."); return
        try:
            age = int(age) if age else None
            if age is not None and not 1 <= age <= 120: raise ValueError
```

```
            if pw:
                run_sql("UPDATE passengers SET
username=%s,password=%s,passenger_name=%s,age=%s WHERE passenger_id=%s",
(user,pw,name,age,self.selected_passenger))
            else:
                run_sql("UPDATE passengers SET username=%s,passenger_name=%s,age=%s
WHERE passenger_id=%s", (user,name,age,self.selected_passenger))
            self.clear_p(); self.refresh_passengers(); messagebox.showinfo("Updated",
"Passenger updated.")
        except ValueError: messagebox.showwarning("Age", "Age must be between 1 and
120.")
        except Error as e: self.fail(e)

    def delete_passenger(self):
        if not self.selected_passenger: messagebox.showwarning("Select", "Select a passenger
row first."); return
        if not messagebox.askyesno("Confirm", "Delete this passenger? Existing bookings
prevent deletion."): return
        try:
            run_sql("DELETE FROM passengers WHERE passenger_id=%s",
(self.selected_passenger,))
            self.clear_p(); self.refresh_passengers(); messagebox.showinfo("Deleted", "Passenger
deleted.")
        except Error as e: self.fail(e)

    # ---------- Train CRUD ----------
    def trains_page(self):
        self.clear(); self.heading("Train Management", "Manage train numbers, names and seat
capacity.")
        form = tk.Frame(self.main, bg=PANEL, padx=14, pady=13); form.pack(fill="x",
padx=25, pady=(0, 10))
        self.t_no, self.t_name, self.t_seats = [self.entry(form, w) for w in (12, 24, 12)]
        for i, (title, widget) in enumerate([("Train number", self.t_no), ("Train name",
self.t_name), ("Total seats", self.t_seats)]):
            self.label(form, title).grid(row=0, column=i, sticky="w", padx=5, pady=(0, 4));
widget.grid(row=1, column=i, padx=5, sticky="ew")
        a = tk.Frame(form, bg=PANEL); a.grid(row=2, column=0, columnspan=3, sticky="w",
pady=(11, 0))
        self.button(a, "＋ Add Train", self.add_train, GREEN).pack(side="left", padx=(0, 6))
        self.button(a, "Update Selected", self.update_train).pack(side="left", padx=6)
        self.button(a, "Delete Selected", self.delete_train, RED).pack(side="left", padx=6)
        self.button(a, "Clear", self.clear_t, PANEL2).pack(side="left", padx=6)
        self.ttree = self.tree(("no", "name", "total", "available"), [("no", "Train No.", 120),
("name", "Train Name", 270), ("total", "Total Seats", 120), ("available", "Available Seats",
150)])
        self.ttree.bind("<<TreeviewSelect>>", self.select_train); self.refresh_trains()

    def refresh_trains(self):
        if not hasattr(self, "ttree") or not self.ttree.winfo_exists(): return
        self.ttree.delete(*self.ttree.get_children())
```

```
        for r in rows("SELECT train_no,train_name,total_seats FROM trains ORDER BY train_no"):
            self.ttree.insert("", "end", values=(r["train_no"], r["train_name"], r["total_seats"], seats_left(r["train_no"])))

    def select_train(self, _=None):
        s = self.ttree.selection()
        if not s: return
        v = self.ttree.item(s[0], "values"); self.selected_train = int(v[0]); self.clear_t(False)
        self.t_no.insert(0, v[0]); self.t_name.insert(0, v[1]); self.t_seats.insert(0, v[2])

    def clear_t(self, reset=True):
        for w in (self.t_no, self.t_name, self.t_seats): w.delete(0, "end")
        if reset: self.selected_train = None

    def add_train(self):
        try:
            no, name, seats = int(self.t_no.get()), self.t_name.get().strip(), int(self.t_seats.get())
            if no <= 0 or seats <= 0 or not name: raise ValueError
            run_sql("INSERT INTO trains VALUES(%s,%s,%s)", (no,name,seats))
            self.clear_t(); self.refresh_trains(); messagebox.showinfo("Added", "Train added.")
        except ValueError: messagebox.showwarning("Input", "Enter a positive train number, train name and seat count.")
        except Error as e: self.fail(e)

    def update_train(self):
        if self.selected_train is None: messagebox.showwarning("Select", "Select a train row first."); return
        try:
            no, name, seats = int(self.t_no.get()), self.t_name.get().strip(), int(self.t_seats.get())
            if no <= 0 or seats <= 0 or not name: raise ValueError
            booked = rows("SELECT COALESCE(SUM(seats),0) n FROM bookings WHERE train_no=%s AND booking_status='BOOKED'", (self.selected_train,))[0]["n"]
            if seats < booked: messagebox.showwarning("Capacity", f"Capacity cannot be below {booked} active booked seats."); return
            run_sql("UPDATE trains SET train_no=%s,train_name=%s,total_seats=%s WHERE train_no=%s", (no,name,seats,self.selected_train))
            self.clear_t(); self.refresh_trains(); messagebox.showinfo("Updated", "Train updated.")
        except ValueError: messagebox.showwarning("Input", "Enter valid positive values.")
        except Error as e: self.fail(e)

    def delete_train(self):
        if self.selected_train is None: messagebox.showwarning("Select", "Select a train row first."); return
        if not messagebox.askyesno("Confirm", "Delete this train? Existing booking history prevents deletion."): return
        try:
            run_sql("DELETE FROM trains WHERE train_no=%s", (self.selected_train,))
            self.clear_t(); self.refresh_trains(); messagebox.showinfo("Deleted", "Train deleted.")
```

```
        except Error as e: self.fail(e)

    # ---------- Booking ----------
    def booking_page(self):
        self.clear(); self.heading("Book a Ticket", "Choose a passenger and train; availability is checked before booking.")
        f = tk.Frame(self.main, bg=PANEL, padx=18, pady=18); f.pack(fill="x", padx=25, pady=(0, 15))
        self.bp = ttk.Combobox(f, state="readonly", width=34)
        self.bt = ttk.Combobox(f, state="readonly", width=34)
        self.bs = self.entry(f, 10)
        for i, (title, widget) in enumerate([("Passenger", self.bp), ("Train", self.bt), ("Seats to book", self.bs)]):
            self.label(f, title).grid(row=0, column=i, sticky="w", padx=6, pady=(0, 5)); widget.grid(row=1, column=i, padx=6, sticky="ew")
        self.bt.bind("<<ComboboxSelected>>", self.check_availability)
        self.button(f, "Check Availability", self.check_availability, PANEL2).grid(row=2, column=0, sticky="w", padx=6, pady=(13, 0))
        self.button(f, "Book Ticket", self.create_booking, GREEN).grid(row=2, column=1, sticky="w", padx=6, pady=(13, 0))
        self.avail_label = tk.Label(f, text="Select a train to view available seats.", bg=PANEL, fg=YELLOW, font=("Segoe UI", 10, "bold"))
        self.avail_label.grid(row=3, column=0, columnspan=3, sticky="w", padx=6, pady=(13, 0))
        self.refresh_booking_choices()

    def refresh_booking_choices(self):
        ps = rows("SELECT passenger_id,passenger_name,username FROM passengers ORDER BY passenger_name")
        ts = rows("SELECT train_no,train_name FROM trains ORDER BY train_no")
        self.bp["values"] = [f'{p["passenger_id"]} | {p["passenger_name"]} ({p["username"]})' for p in ps]
        self.bt["values"] = [f'{t["train_no"]} | {t["train_name"]}' for t in ts]

    def check_availability(self, _=None):
        if not self.bt.get(): self.avail_label.config(text="Select a train to view available seats.", fg=YELLOW); return
        try:
            no = int(self.bt.get().split("|")[0].strip()); n = seats_left(no)
            self.avail_label.config(text=f"Available seats: {n}", fg=GREEN if n and n > 0 else RED)
        except Error as e: self.fail(e)

    def create_booking(self):
        if not self.bp.get() or not self.bt.get() or not self.bs.get().strip():
            messagebox.showwarning("Missing", "Select a passenger, train and number of seats."); return
        conn = cur = None
        try:
```

```
            pid = int(self.bp.get().split("|")[0].strip()); tno = int(self.bt.get().split("|")[0].strip());
count = int(self.bs.get())
            if count <= 0: raise ValueError("Seats must be a positive whole number.")
            conn = connect_db(); conn.start_transaction()
            cur = conn.cursor()
            cur.execute("SELECT total_seats FROM trains WHERE train_no=%s FOR
UPDATE", (tno,))
            train = cur.fetchone()
            if not train: raise ValueError("Train not found.")
            cur.execute("SELECT COALESCE(SUM(seats),0) FROM bookings WHERE
train_no=%s AND booking_status='BOOKED'", (tno,))
            booked = cur.fetchone()[0]; remaining = train[0] - booked
            if count > remaining: raise ValueError(f"Only {remaining} seat(s) available.")
            cur.execute("INSERT INTO bookings(passenger_id,train_no,seats,booking_status)
VALUES(%s,%s,%s,'BOOKED')", (pid,tno,count))
            bid = cur.lastrowid; conn.commit()
            self.bs.delete(0, "end"); self.check_availability()
            messagebox.showinfo("Booking confirmed", f"Ticket booked successfully!\nBooking
ID: {bid}\nSeats: {count}")
        except ValueError as e:
            if conn: conn.rollback()
            messagebox.showwarning("Cannot book", str(e))
        except Error as e:
            if conn: conn.rollback()
            self.fail(e)
        finally:
            if cur: cur.close()
            if conn: conn.close()

    # ---------- Booking history / cancellation ----------
    def bookings_page(self):
        self.clear(); self.heading("Booking Management", "Search reservations and cancel active
tickets.")
        bar = tk.Frame(self.main, bg=PANEL, padx=12, pady=12); bar.pack(fill="x", padx=25,
pady=(0, 10))
        self.search = self.entry(bar, 25); self.search.grid(row=0, column=0, padx=5)
        self.button(bar, "Search", self.refresh_bookings).grid(row=0, column=1, padx=5)
        self.button(bar, "Show All", self.show_all_bookings, PANEL2).grid(row=0, column=2,
padx=5)
        self.button(bar, "Cancel Selected", self.cancel_booking, RED).grid(row=0, column=3,
padx=5)
        self.btree = self.tree(("id", "passenger", "no", "train", "seats", "status"),
            [("id", "Booking ID", 90), ("passenger", "Passenger", 190), ("no", "Train No.", 100),
("train", "Train", 220), ("seats", "Seats", 75), ("status", "Status", 120)])
        self.btree.bind("<<TreeviewSelect>>", self.select_booking); self.refresh_bookings()

    def show_all_bookings(self):
        self.search.delete(0, "end"); self.refresh_bookings()

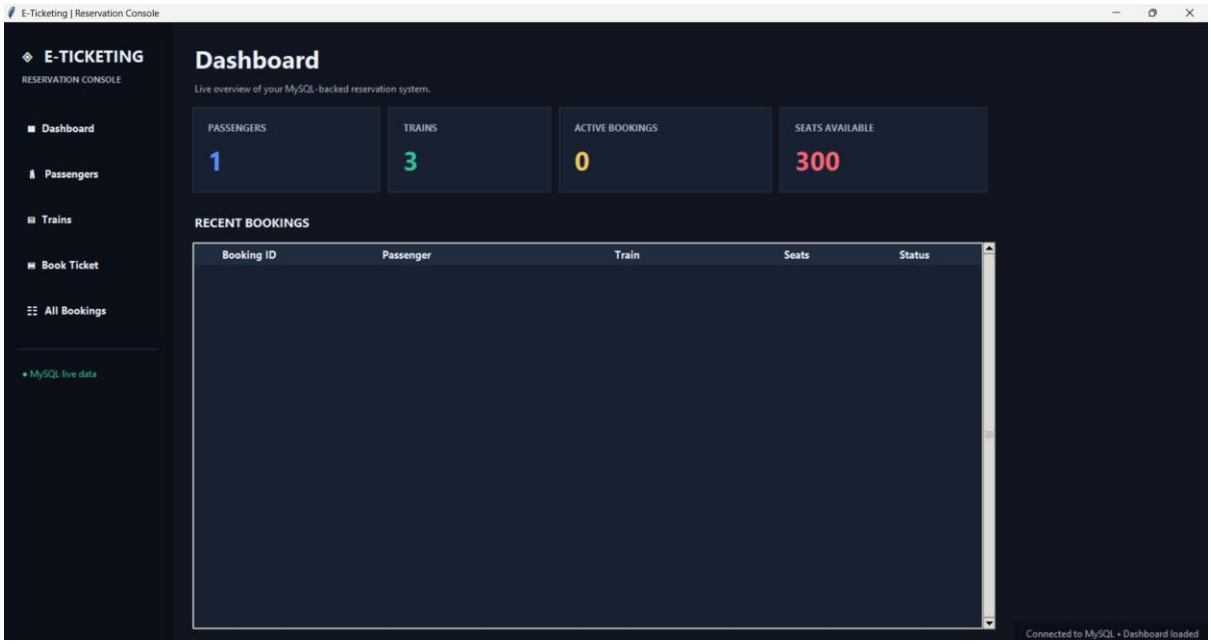
    def refresh_bookings(self):
```

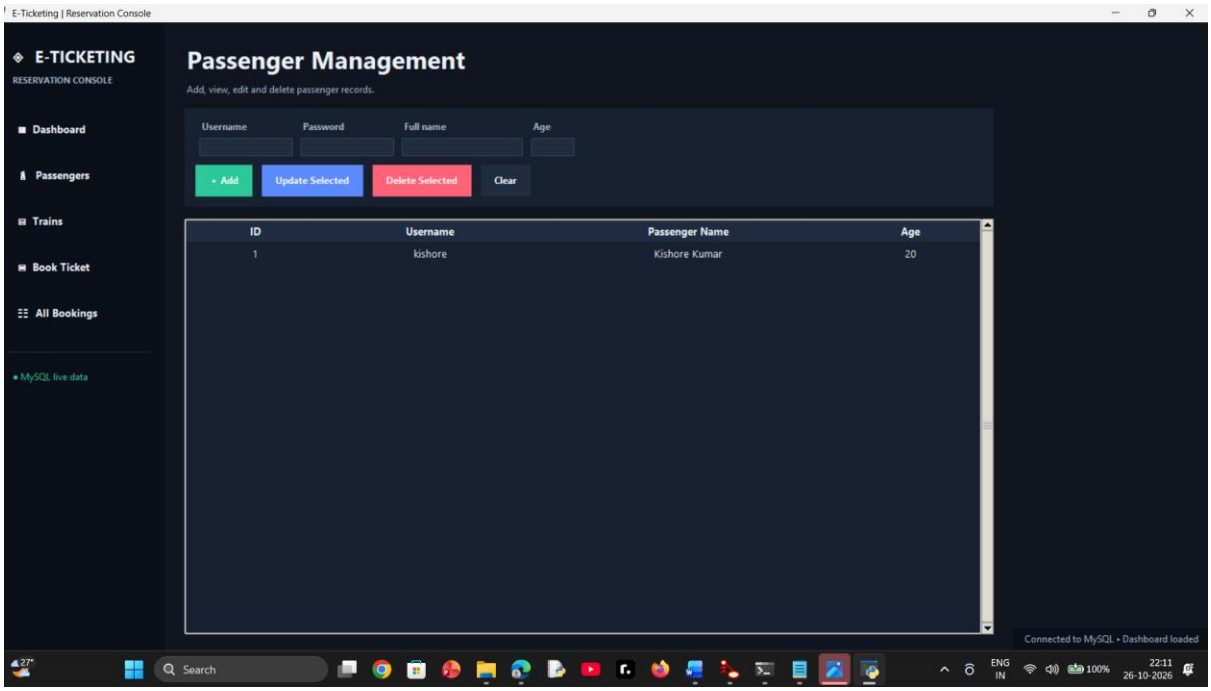
```
        if not hasattr(self, "btree") or not self.btree.winfo_exists(): return
        self.btree.delete(*self.btree.get_children())
        term = self.search.get().strip() if hasattr(self, "search") else ""
        sql = """SELECT
b.booking_id,p.passenger_name,b.train_no,t.train_name,b.seats,b.booking_status
                FROM bookings b JOIN passengers p ON p.passenger_id=b.passenger_id
                JOIN trains t ON t.train_no=b.train_no"""
        params = ()
        if term:
            sql += " WHERE CAST(b.booking_id AS CHAR) LIKE %s OR p.passenger_name
LIKE %s OR t.train_name LIKE %s"
            params = (f"%{term}%", f"%{term}%", f"%{term}%")
        sql += " ORDER BY b.booking_id DESC"
        for r in rows(sql, params):
            self.btree.insert("", "end", values=(r["booking_id"], r["passenger_name"],
r["train_no"], r["train_name"], r["seats"], r["booking_status"]))

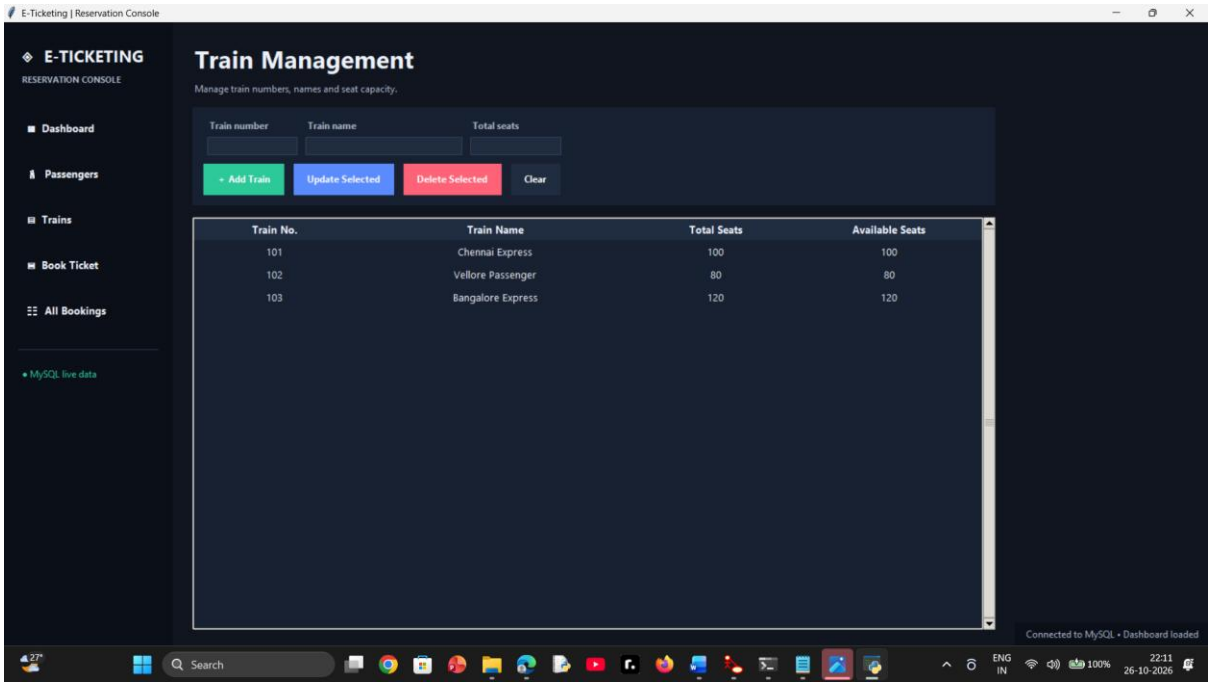
    def select_booking(self, _=None):
        s = self.btree.selection()
        if s: self.selected_booking = int(self.btree.item(s[0], "values")[0])

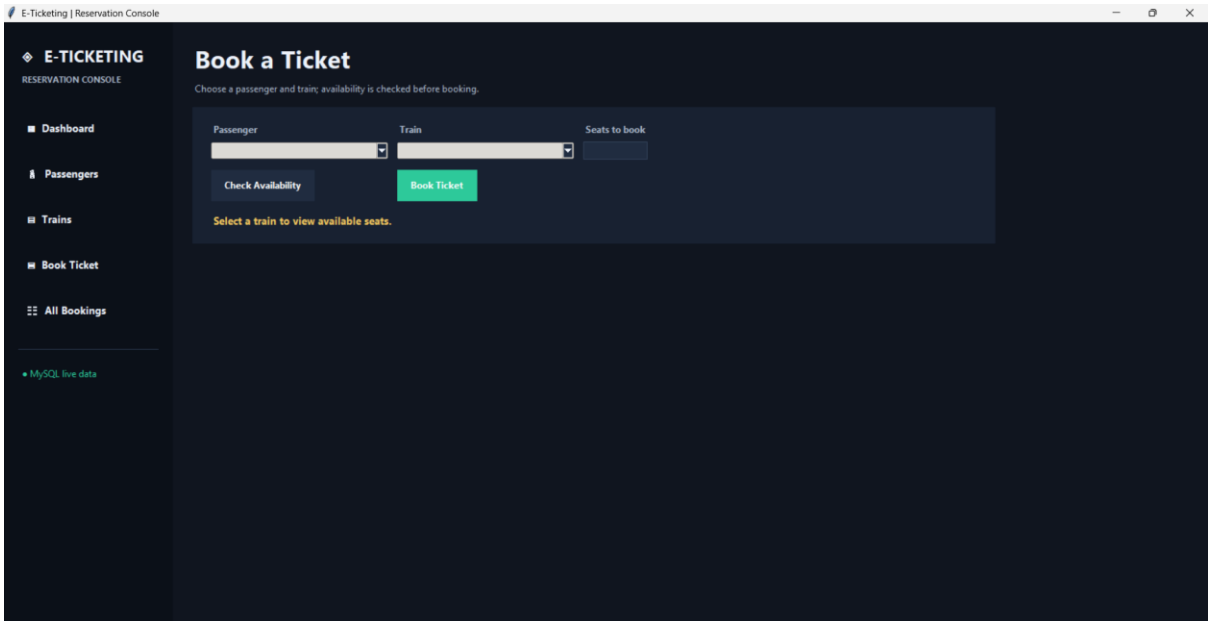
    def cancel_booking(self):
        if not self.selected_booking: messagebox.showwarning("Select", "Select a booking row
first."); return
        r = rows("SELECT booking_status FROM bookings WHERE booking_id=%s",
(self.selected_booking,))
        if not r or r[0]["booking_status"] != "BOOKED":
            messagebox.showinfo("Info", "This booking is already cancelled."); return
        if not messagebox.askyesno("Confirm cancellation", f"Cancel booking
#{self.selected_booking}?"): return
        try:
            run_sql("UPDATE bookings SET booking_status='CANCELLED' WHERE
booking_id=%s", (self.selected_booking,))
            self.selected_booking = None; self.refresh_bookings()
            messagebox.showinfo("Cancelled", "Ticket cancelled. Seats are available again.")
        except Error as e: self.fail(e)


if __name__ == "__main__":
    try:
        setup_database()
        App().mainloop()
    except Error as exc:
        messagebox.showerror("MySQL connection failed",
            f"Could not connect to database 'eticketing'.\n\n{exc}\n\n"
            "Edit DB_CONFIG in this file with your MySQL password and confirm MySQL is
running.")
```


E-Ticketing | Reservation Console
E-TICKETING
RESERVATION CONSOLE
Dashboard
Passengers
Trains
Book Ticket
All Bookings
MySQL live data
Dashboard
Live overview of your MySQL-backed reservation system.
PASSENGERS
1
TRAINS
3
ACTIVE BOOKINGS
0
SEATS AVAILABLE
300
RECENT BOOKINGS
Booking ID
Passenger
Train
Seats
Status
Connected to MySQL • Dashboard loaded


E-Ticketing | Reservation Console
E-TICKETING
RESERVATION CONSOLE
Dashboard
Passengers
Trains
Book Ticket
All Bookings
MySQL live data
Passenger Management
Add, view, edit and delete passenger records.
Username
Password
Full name
Age
+ Add
Update Selected
Delete Selected
Clear
ID
Username
Passenger Name
Age
1
kishore
Kishore Kumar
20
Connected to MySQL • Dashboard loaded
Search
ENG
IN
22:11
26-10-2026


E-Ticketing | Reservation Console
E-TICKETING
RESERVATION CONSOLE
Dashboard
Passengers
Trains
Book Ticket
All Bookings
MySQL live data
Train Management
Manage train numbers, names and seat capacity.
Train number
Train name
Total seats
+ Add Train
Update Selected
Delete Selected
Clear
Train No.
Train Name
Total Seats
Available Seats
101
Chennai Express
100
100
102
Vellore Passenger
80
80
103
Bangalore Express
120
120
Connected to MySQL • Dashboard loaded
Search
ENG
IN
22:11
26-10-2026


E-Ticketing | Reservation Console
E-TICKETING
RESERVATION CONSOLE
Dashboard
Passengers
Trains
Book Ticket
All Bookings
MySQL live data
Book a Ticket
Choose a passenger and train; availability is checked before booking.
Passenger
Train
Seats to book
Check Availability
Book Ticket
Select a train to view available seats.